Database Management Systems Lab

Subject Code: 23A05402P

SQL, PL/SQL and JDBC Programs for All Experiments

Each experiment contains the aim, the complete program and the sample output for the data used in the programs.

List of Experiments

Experiment 1 - Creation, altering and dropping of tables and inserting rows into a table (using constraints while creating tables); examples using the SELECT command.

Experiment 2 - Queries (along with sub-queries) using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, INTERSECT and constraints. Example: select the roll number and name of the student who secured fourth rank in the class.

Experiment 3 - Queries using aggregate functions (COUNT, SUM, AVG, MAX and MIN), GROUP BY, HAVING, and creation and dropping of views.

Experiment 4 - Queries using conversion functions (to_char, to_number, to_date), string functions (concatenation, lpad, rpad, ltrim, rtrim, lower, upper, initcap, length, substr, instr) and date functions (sysdate, next_day, add_months, last_day, months_between, least, greatest, trunc, round, to_char, to_date).

Experiment 5 - (i) A simple PL/SQL program with declaration, executable and exception-handling sections. (ii) Insert data into the student table and use COMMIT, ROLLBACK and SAVEPOINT in a PL/SQL block.

Experiment 6 - A program that uses NESTED IF, CASE and CASE expressions, extended with the NULLIF and COALESCE functions.

Experiment 7 - Programs using WHILE loops, numeric FOR loops, nested loops, error handling, built-in exceptions, user-defined exceptions and RAISE_APPLICATION_ERROR.

Experiment 8 - Programs using the creation of procedures, passing parameters IN and OUT of procedures.

Experiment 9 - Programs using the creation of stored functions, invoking functions in SQL statements, and writing complex functions.

Experiment 10 - Programs using parameters in a CURSOR, FOR UPDATE cursor, WHERE CURRENT OF clause and cursor variables.

Experiment 11 - Programs using BEFORE and AFTER triggers, row and statement triggers, and INSTEAD OF triggers.

Experiment 12 - Create a table and perform the search operation on the table using indexing and non-indexing techniques.

Experiment 13 - Write a Java program that connects to a database using JDBC.

Experiment 14 - Write a Java program to connect to a database using JDBC and insert values into it.

Experiment 15 - Write a Java program to connect to a database using JDBC and delete values from it.

Before You Start

Software: Oracle Database (11g / 12c / XE) with SQL*Plus or SQL Developer. The JDBC programs also need the Oracle JDBC driver (ojdbc8.jar) and the JDK.

Order: run Experiment 1 first. It creates the DEPARTMENT and STUDENT tables with eight sample students (roll numbers 101 to 108) that the other experiments use. Experiments that change the data (5, 8, 10, 11) roll back or clean up after themselves, so the outputs shown stay the same.

PL/SQL: type `SET SERVEROUTPUT ON` in SQL*Plus before running a block so that DBMS_OUTPUT messages are displayed. The `/` on a line of its own executes the block.

Sample data (STUDENT):

```
ROLL_NO  NAME    DEPT_ID  MARKS
101      Asha    10       88
102      Ravi    20       92
103      Meena   10       79
104      Kiran   30       65
105      Sita    30       90
106      Arjun   20       73
107      Divya   10       85
108      Kumar   30       95
```

DEPARTMENT: 10 = CSE, 20 = ECE, 30 = AIML.

Note: the outputs are the results expected for this sample data. Items such as SYSDATE, execution time and constraint names depend on the machine on which the program is run.

Experiment 1

Aim: Creation, altering and dropping of tables and inserting rows into a table (using constraints while creating tables); examples using the SELECT command.

Program

```
-- Experiment 1: CREATE, ALTER, DROP tables, INSERT rows and SELECT
-- (tables are created with constraints)

-- 1. CREATE TABLE with constraints
CREATE TABLE department (
    dept_id    NUMBER(3)    PRIMARY KEY,
    dept_name  VARCHAR2(30) NOT NULL UNIQUE
);

CREATE TABLE student (
    roll_no    NUMBER(5)    PRIMARY KEY,
    name       VARCHAR2(30) NOT NULL,
    dept_id    NUMBER(3)    REFERENCES department(dept_id),
    marks      NUMBER(3)    CHECK (marks BETWEEN 0 AND 100),
    email      VARCHAR2(40) UNIQUE,
    join_date  DATE         DEFAULT SYSDATE
);

DESC student;

-- 2. INSERT rows
INSERT INTO department VALUES (10, 'CSE');
INSERT INTO department VALUES (20, 'ECE');
INSERT INTO department VALUES (30, 'AIML');

INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (101, 'Asha', 10, 88);
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (102, 'Ravi', 20, 92);
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (103, 'Meena', 10, 79);
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (104, 'Kiran', 30, 65);
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (105, 'Sita', 30, 90);
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (106, 'Arjun', 20, 73);
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (107, 'Divya', 10, 85);
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (108, 'Kumar', 30, 95);
COMMIT;

-- 3. SELECT examples
SELECT * FROM department;
SELECT roll_no, name, marks FROM student;
```

```
SELECT name, marks FROM student WHERE marks > 80 ORDER BY marks DESC;
SELECT DISTINCT dept_id FROM student;

-- 4. ALTER TABLE
ALTER TABLE student ADD (phone VARCHAR2(10));          -- add a column
ALTER TABLE student MODIFY (name VARCHAR2(40));        -- change size
ALTER TABLE student RENAME COLUMN phone TO mobile;     -- rename column
ALTER TABLE student ADD CONSTRAINT chk_mobile CHECK (LENGTH(mobile) = 10);
ALTER TABLE student DROP COLUMN mobile;                -- drop a column

-- 5. Constraint violation examples
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (101, 'Test', 10, 50);
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (109, 'Test', 10, 150);
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (109, 'Test', 99, 50);

-- 6. DROP TABLE (use a scratch table so the sample data is kept)
CREATE TABLE temp_demo (id NUMBER);
DROP TABLE temp_demo;
```

Output

```
SQL> DESC student;
 Name         Null?     Type
 ------------ --------- ------------
 ROLL_NO      NOT NULL  NUMBER(5)
 NAME         NOT NULL  VARCHAR2(30)
 DEPT_ID                NUMBER(3)
 MARKS                  NUMBER(3)
 EMAIL                  VARCHAR2(40)
 JOIN_DATE              DATE

SQL> SELECT * FROM department;
   DEPT_ID DEPT_NAME
---------- ------------------------------
        10 CSE
        20 ECE
        30 AIML

SQL> SELECT name, marks FROM student WHERE marks > 80 ORDER BY marks DESC;
NAME                 MARKS
---------------- --------
Kumar                  95
Ravi                   92
Sita                   90
Asha                   88
Divya                  85

SQL> SELECT DISTINCT dept_id FROM student;
   DEPT_ID
----------
```

```
        10
        20
        30

Constraint violations:
ORA-00001: unique constraint (SYS_C0012345) violated
ORA-02290: check constraint (SYS_C0012346) violated
ORA-02291: integrity constraint (SYS_C0012347) violated - parent key not found

Table dropped.
```

Experiment 2

Aim: Queries (along with sub-queries) using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, INTERSECT and constraints. Example: select the roll number and name of the student who secured fourth rank in the class.

Program

```
-- Experiment 2: Queries and sub-queries (ANY, ALL, IN, EXISTS, NOT EXISTS,
--               UNION, INTERSECT) and constraints
-- Two extra tables are used for set operations
CREATE TABLE sports  (roll_no NUMBER(5), sport_name VARCHAR2(20));
CREATE TABLE library (roll_no NUMBER(5));
INSERT INTO sports  VALUES (101, 'Cricket');
INSERT INTO sports  VALUES (103, 'Chess');
INSERT INTO sports  VALUES (105, 'Cricket');
INSERT INTO sports  VALUES (108, 'Chess');
INSERT INTO library VALUES (101);
INSERT INTO library VALUES (102);
INSERT INTO library VALUES (105);
COMMIT;

-- 1. Roll number and name of the student who secured the FOURTH rank
SELECT roll_no, name
FROM   student s
WHERE  3 = (SELECT COUNT(DISTINCT marks)
            FROM   student
            WHERE  marks > s.marks);

-- 2. IN : students of CSE or ECE
SELECT roll_no, name FROM student
WHERE  dept_id IN (SELECT dept_id FROM department
                   WHERE dept_name IN ('CSE', 'ECE'))
ORDER BY roll_no;

-- 3. ANY : marks greater than ANY CSE student's marks
SELECT name, marks FROM student
WHERE  marks > ANY (SELECT marks FROM student WHERE dept_id = 10)
ORDER BY marks;

-- 4. ALL : marks greater than ALL ECE students' marks
SELECT name, marks FROM student
WHERE  marks > ALL (SELECT marks FROM student WHERE dept_id = 20);

-- 5. EXISTS : students who play some sport
SELECT roll_no, name FROM student s
WHERE  EXISTS (SELECT 1 FROM sports sp WHERE sp.roll_no = s.roll_no)
ORDER BY roll_no;

-- 6. NOT EXISTS : students who play no sport
SELECT roll_no, name FROM student s
WHERE  NOT EXISTS (SELECT 1 FROM sports sp WHERE sp.roll_no = s.roll_no)
```

```
ORDER BY roll_no;

-- 7. UNION : students who play a sport OR use the library
SELECT roll_no FROM sports
UNION
SELECT roll_no FROM library;

-- 8. INTERSECT : students who play a sport AND use the library
SELECT roll_no FROM sports
INTERSECT
SELECT roll_no FROM library;

-- 9. MINUS : students who play a sport but do not use the library
SELECT roll_no FROM sports
MINUS
SELECT roll_no FROM library;

-- 10. Constraints on the new tables
ALTER TABLE sports ADD CONSTRAINT pk_sports PRIMARY KEY (roll_no, sport_name);
ALTER TABLE sports ADD CONSTRAINT fk_sports_student
      FOREIGN KEY (roll_no) REFERENCES student(roll_no);
INSERT INTO sports VALUES (999, 'Chess');
```

Output

```
Query 1 (fourth rank):
   ROLL_NO NAME
---------- ----------
       101 Asha

Query 2 (IN):
   ROLL_NO NAME
---------- ----------
       101 Asha
       102 Ravi
       103 Meena
       106 Arjun
       107 Divya

Query 3 (ANY - marks greater than the lowest CSE mark, 79):
NAME            MARKS
---------- ----------
Divya              85
Asha               88
Sita               90
Ravi               92
Kumar              95

Query 4 (ALL - marks greater than the highest ECE mark, 92):
NAME            MARKS
---------- ----------
Kumar              95
Query 5 (EXISTS):
```

```
   ROLL_NO NAME
---------- ----------
       101 Asha
       103 Meena
       105 Sita
       108 Kumar

Query 6 (NOT EXISTS):
   ROLL_NO NAME
---------- ----------
       102 Ravi
       104 Kiran
       106 Arjun
       107 Divya

Query 7 (UNION):      101, 102, 103, 105, 108
Query 8 (INTERSECT):  101, 105
Query 9 (MINUS):      103, 108

Query 10 (foreign key violation):
ORA-02291: integrity constraint violated - parent key not found
```

Experiment 3

Aim: Queries using aggregate functions (COUNT, SUM, AVG, MAX and MIN), GROUP BY, HAVING, and creation and dropping of views.

Program

```
-- Experiment 3: Aggregate functions, GROUP BY, HAVING, and Views

-- 1. Aggregate functions
SELECT COUNT(*)            AS total_students,
       SUM(marks)          AS total_marks,
       ROUND(AVG(marks),2) AS average_marks,
       MAX(marks)          AS highest,
       MIN(marks)          AS lowest
FROM   student;

-- 2. COUNT of a column ignores NULL values
SELECT COUNT(email) AS students_with_email FROM student;

-- 3. GROUP BY : department-wise statistics
SELECT dept_id,
       COUNT(*)            AS no_of_students,
       SUM(marks)          AS total_marks,
       ROUND(AVG(marks),2) AS avg_marks
FROM   student
GROUP BY dept_id
ORDER BY dept_id;

-- 4. HAVING : departments whose average marks are above 83
SELECT dept_id, ROUND(AVG(marks),2) AS avg_marks
FROM   student
GROUP BY dept_id
HAVING AVG(marks) > 83
ORDER BY dept_id;

-- 5. GROUP BY with a join : department names
SELECT d.dept_name, MAX(s.marks) AS top_marks
FROM   student s JOIN department d ON s.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY d.dept_name;

-- 6. Creating a view
CREATE VIEW topper_view AS
    SELECT roll_no, name, marks
    FROM   student
    WHERE  marks >= 85;

SELECT * FROM topper_view ORDER BY roll_no;

-- 7. A view with aggregate functions
CREATE OR REPLACE VIEW dept_summary AS
    SELECT dept_id, COUNT(*) AS students, ROUND(AVG(marks),2) AS avg_marks
```

```
    FROM   student
    GROUP BY dept_id;

SELECT * FROM dept_summary;

-- 8. Dropping views
DROP VIEW topper_view;
DROP VIEW dept_summary;
```

Output

```
TOTAL_STUDENTS TOTAL_MARKS AVERAGE_MARKS HIGHEST LOWEST
-------------- ----------- ------------- ------- ------
             8         667         83.38      95     65

STUDENTS_WITH_EMAIL
-------------------
                  0

   DEPT_ID NO_OF_STUDENTS TOTAL_MARKS AVG_MARKS
---------- -------------- ----------- ---------
        10              3         252        84
        20              2         165      82.5
        30              3         250     83.33

HAVING AVG(marks) > 83:
   DEPT_ID AVG_MARKS
---------- ---------
        10        84
        30     83.33

DEPT_NAME        TOP_MARKS
---------------- ---------
AIML                    95
CSE                     88
ECE                     92

SELECT * FROM topper_view:
   ROLL_NO NAME          MARKS
---------- ---------- --------
       101 Asha             88
       102 Ravi             92
       105 Sita             90
       107 Divya            85
       108 Kumar            95

View dropped.
```

Experiment 4

Aim: Queries using conversion functions (to_char, to_number, to_date), string functions (concatenation, lpad, rpad, ltrim, rtrim, lower, upper, initcap, length, substr, instr) and date functions (sysdate, next_day, add_months, last_day, months_between, least, greatest, trunc, round, to_char, to_date).

Program

```
-- Experiment 4: Conversion, string and date functions
-- (a fixed date, 15-AUG-2024, is used so the results are repeatable)

-- ---------- Conversion functions ----------
SELECT TO_CHAR(DATE '2024-08-15', 'DD FMMonth YYYY')   AS date_text,
       TO_CHAR(DATE '2024-08-15', 'FMDay')             AS day_name,
       TRIM(TO_CHAR(1234.5, '9,999.00'))               AS num_text,
       TO_NUMBER('1234.50')                            AS num1,
       TO_NUMBER('1,234', '9,999')                     AS num2,
       TO_DATE('15-08-2024', 'DD-MM-YYYY')             AS date1
FROM   dual;

-- ---------- String functions ----------
SELECT 'DBMS' || ' ' || 'Lab'              AS concatenation,
       LPAD('DBMS', 8, '*')                AS lpad_demo,
       RPAD('DBMS', 8, '*')                AS rpad_demo,
       LTRIM('   hello')                   AS ltrim_demo,
       RTRIM('hello   ')                   AS rtrim_demo,
       LOWER('DBMS Lab')                   AS lower_demo,
       UPPER('dbms lab')                   AS upper_demo,
       INITCAP('database management system') AS initcap_demo
FROM   dual;

SELECT LENGTH('Database')             AS len,
       SUBSTR('Database', 1, 4)       AS sub1,
       SUBSTR('Database', -4)         AS sub2,
       INSTR('Database', 'a')         AS first_a,
       INSTR('Database', 'a', 1, 2)   AS second_a,
       INSTR('Database', 'a', 1, 3)   AS third_a
FROM   dual;

-- Using string functions on the table
SELECT UPPER(name) AS name_upper, LENGTH(name) AS name_length,
       SUBSTR(name, 1, 3) AS first3
FROM   student
WHERE  roll_no <= 103;

-- ---------- Date functions ----------
SELECT SYSDATE FROM dual;                    -- current date and time

SELECT NEXT_DAY(DATE '2024-08-15', 'MONDAY')       AS next_monday,
       ADD_MONTHS(DATE '2024-08-15', 3)            AS after_3_months,
       LAST_DAY(DATE '2024-08-15')                 AS month_end,
```

```
       MONTHS_BETWEEN(DATE '2024-08-15',
                      DATE '2024-02-15')           AS months_diff
FROM   dual;

SELECT LEAST(5, 12, 3)        AS least_num,
       GREATEST(5, 12, 3)     AS greatest_num,
       LEAST(DATE '2024-01-01', DATE '2024-08-15') AS earlier_date
FROM   dual;

SELECT TRUNC(DATE '2024-08-15', 'MM')    AS trunc_month,
       TRUNC(DATE '2024-08-15', 'YYYY')  AS trunc_year,
       ROUND(DATE '2024-08-20', 'MM')    AS round_month,
       ROUND(DATE '2024-08-15', 'YYYY')  AS round_year
FROM   dual;

-- Date functions on the table
SELECT name, TO_CHAR(join_date, 'DD-MON-YYYY') AS joined,
       ADD_MONTHS(join_date, 6)                AS review_date
FROM   student
WHERE  roll_no = 101;
```

Output

```
Conversion functions:
DATE_TEXT       DAY_NAME  NUM_TEXT  NUM1  NUM2  DATE1
--------------- --------- --------- ----- ----- ---------
15 August 2024  Thursday  1,234.50  1234.5 1234 15-AUG-24

String functions:
CONCATENATION  LPAD_DEMO  RPAD_DEMO  LTRIM_DEMO  RTRIM_DEMO
-------------  ---------  ---------  ----------  ----------
DBMS Lab       ****DBMS   DBMS****   hello       hello

LOWER_DEMO  UPPER_DEMO  INITCAP_DEMO
----------  ----------  ----------------------------
dbms lab    DBMS LAB    Database Management System

LEN  SUB1  SUB2  FIRST_A  SECOND_A  THIRD_A
---  ----  ----  -------  --------  -------
  8  Data  base         2         4        6

Date functions:
NEXT_MONDAY  AFTER_3_MONTHS  MONTH_END  MONTHS_DIFF
-----------  --------------  ---------  -----------
19-AUG-24    15-NOV-24       31-AUG-24            6

LEAST_NUM  GREATEST_NUM  EARLIER_DATE
---------  ------------  ------------
        3            12  01-JAN-24

TRUNC_MONTH  TRUNC_YEAR  ROUND_MONTH  ROUND_YEAR
-----------  ----------  -----------  ----------
01-AUG-24    01-JAN-24   01-SEP-24    01-JAN-25
```

(SYSDATE and the join_date query show the date on which the program is run.)

Experiment 5

Aim: (i) A simple PL/SQL program with declaration, executable and exception-handling sections. (ii) Insert data into the student table and use COMMIT, ROLLBACK and SAVEPOINT in a PL/SQL block.

Program

```
-- Experiment 5(i): PL/SQL block with declaration, executable and
--                  exception-handling sections
SET SERVEROUTPUT ON

DECLARE
    -- Declaration section
    CURSOR c_first IS
        SELECT roll_no, name, marks
        FROM   student
        WHERE  marks >= 60 AND marks < 75      -- first class
        ORDER BY roll_no;
    v_count       NUMBER := 0;
    e_no_records  EXCEPTION;                   -- user-defined exception
BEGIN
    -- Executable section
    DBMS_OUTPUT.PUT_LINE('Students who secured First Class:');
    FOR rec IN c_first LOOP
        v_count := v_count + 1;
        DBMS_OUTPUT.PUT_LINE('Roll: ' || rec.roll_no || '  Name: ' ||
                             rec.name || '  Marks: ' || rec.marks);
    END LOOP;

    IF v_count = 0 THEN
        RAISE e_no_records;
    END IF;
    DBMS_OUTPUT.PUT_LINE('Total first class students: ' || v_count);
EXCEPTION
    -- Exception-handling section
    WHEN e_no_records THEN
        DBMS_OUTPUT.PUT_LINE('No records were found for first class.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

-- Experiment 5(ii): COMMIT, ROLLBACK and SAVEPOINT in a PL/SQL block
DECLARE
    v_count NUMBER;
BEGIN
    INSERT INTO student (roll_no, name, dept_id, marks)
        VALUES (109, 'Latha', 10, 70);
    SAVEPOINT sp1;

    INSERT INTO student (roll_no, name, dept_id, marks)
        VALUES (110, 'Mohan', 20, 81);
    SAVEPOINT sp2;
```

```
    INSERT INTO student (roll_no, name, dept_id, marks)
        VALUES (111, 'Nisha', 30, 77);

    SELECT COUNT(*) INTO v_count FROM student;
    DBMS_OUTPUT.PUT_LINE('After 3 inserts           : ' || v_count);

    ROLLBACK TO sp2;                -- undo only the third insert
    SELECT COUNT(*) INTO v_count FROM student;
    DBMS_OUTPUT.PUT_LINE('After ROLLBACK TO sp2     : ' || v_count);

    ROLLBACK TO sp1;                -- undo the second insert as well
    SELECT COUNT(*) INTO v_count FROM student;
    DBMS_OUTPUT.PUT_LINE('After ROLLBACK TO sp1     : ' || v_count);

    COMMIT;                         -- make the first insert permanent
    DBMS_OUTPUT.PUT_LINE('Transaction committed.');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Duplicate roll - transaction rolled back.');
END;
/

-- Clean-up so that later experiments use the same 8 sample rows
DELETE FROM student WHERE roll_no = 109;
COMMIT;
```

Output

```
Students who secured First Class:
Roll: 104  Name: Kiran  Marks: 65
Roll: 106  Name: Arjun  Marks: 73
Total first class students: 2

PL/SQL procedure successfully completed.

After 3 inserts           : 11
After ROLLBACK TO sp2     : 10
After ROLLBACK TO sp1     : 9
Transaction committed.

PL/SQL procedure successfully completed.

(If no student is in the range, the output is:
 Students who secured First Class:
 No records were found for first class.)
```

Experiment 6

Aim: A program that uses NESTED IF, CASE and CASE expressions, extended with the NULLIF and COALESCE functions.

Program

```
-- Experiment 6: NESTED IF, CASE statement, CASE expression,
--               NULLIF and COALESCE
SET SERVEROUTPUT ON

DECLARE
    v_marks   NUMBER := 82;
    v_grade   CHAR(1);
    v_result  VARCHAR2(20);
    v_remark  VARCHAR2(20);
BEGIN
    DBMS_OUTPUT.PUT_LINE('Marks : ' || v_marks);

    -- 1. NESTED IF
    IF v_marks >= 35 THEN
        IF v_marks >= 60 THEN
            IF v_marks >= 75 THEN
                v_result := 'Distinction';
            ELSE
                v_result := 'First Class';
            END IF;
        ELSE
            v_result := 'Pass Class';
        END IF;
    ELSE
        v_result := 'Fail';
    END IF;
    DBMS_OUTPUT.PUT_LINE('Result (nested IF)    : ' || v_result);

    -- 2. Searched CASE statement
    CASE
        WHEN v_marks >= 90 THEN v_grade := 'O';
        WHEN v_marks >= 80 THEN v_grade := 'A';
        WHEN v_marks >= 70 THEN v_grade := 'B';
        WHEN v_marks >= 60 THEN v_grade := 'C';
        ELSE v_grade := 'F';
    END CASE;
    DBMS_OUTPUT.PUT_LINE('Grade (CASE)          : ' || v_grade);

    -- 3. Simple CASE statement (selector form)
    CASE v_grade
        WHEN 'O' THEN v_remark := 'Outstanding';
        WHEN 'A' THEN v_remark := 'Very Good';
        WHEN 'B' THEN v_remark := 'Good';
        ELSE v_remark := 'Needs improvement';
    END CASE;
    DBMS_OUTPUT.PUT_LINE('Remark (simple CASE)  : ' || v_remark);
```

```
    -- 4. CASE expression inside an assignment
    v_remark := CASE WHEN v_marks >= 35 THEN 'PASS' ELSE 'FAIL' END;
    DBMS_OUTPUT.PUT_LINE('Status (CASE expr.)   : ' || v_remark);
END;
/

-- 5. CASE expression in a SELECT statement
SELECT name, marks,
       CASE WHEN marks >= 90 THEN 'O'
            WHEN marks >= 80 THEN 'A'
            WHEN marks >= 70 THEN 'B'
            ELSE 'C' END AS grade
FROM   student
ORDER BY roll_no;

-- 6. NULLIF : returns NULL when both arguments are equal
SELECT NULLIF(10, 10) AS same_values,
       NULLIF(10, 20) AS different_values
FROM   dual;

-- 7. COALESCE : returns the first non-NULL argument
SELECT COALESCE(NULL, NULL, 'Default') AS first_not_null FROM dual;

SELECT name, COALESCE(email, 'no email given') AS email_info
FROM   student
WHERE  roll_no <= 3;

-- 8. NULLIF and COALESCE together (avoid divide-by-zero)
SELECT COALESCE(100 / NULLIF(0, 0), -1) AS safe_division FROM dual;
```

Output

```
Marks : 82
Result (nested IF)    : Distinction
Grade (CASE)          : A
Remark (simple CASE)  : Very Good
Status (CASE expr.)   : PASS

NAME            MARKS GRADE
---------- ---------- -----
Asha               88 A
Ravi               92 O
Meena              79 B
Kiran              65 C
Sita               90 O
Arjun              73 B
Divya              85 A
Kumar              95 O

SAME_VALUES DIFFERENT_VALUES
----------- ----------------
                         10
```

```
FIRST_NOT_NULL
--------------
Default

NAME       EMAIL_INFO
---------- ---------------
Asha       no email given
Ravi       no email given
Meena      no email given

SAFE_DIVISION
-------------
           -1
```

Experiment 7

Aim: Programs using WHILE loops, numeric FOR loops, nested loops, error handling, built-in exceptions, user-defined exceptions and RAISE_APPLICATION_ERROR.

Program

```
-- Experiment 7: WHILE loop, numeric FOR loop, nested loops and
--               exception handling (built-in, user-defined,
--               RAISE_APPLICATION_ERROR)
SET SERVEROUTPUT ON

-- 1. WHILE loop : sum of the first 5 numbers
DECLARE
    i   NUMBER := 1;
    sm  NUMBER := 0;
BEGIN
    WHILE i <= 5 LOOP
        sm := sm + i;
        DBMS_OUTPUT.PUT_LINE('i = ' || i || '   running sum = ' || sm);
        i := i + 1;
    END LOOP;
END;
/

-- 2. Numeric FOR loop : multiplication table of 5
BEGIN
    FOR n IN 1..5 LOOP
        DBMS_OUTPUT.PUT_LINE('5 x ' || n || ' = ' || 5 * n);
    END LOOP;
END;
/

-- 3. Nested loops : star triangle
DECLARE
    line_text VARCHAR2(20);
BEGIN
    FOR r IN 1..4 LOOP
        line_text := NULL;
        FOR c IN 1..r LOOP
            line_text := line_text || '* ';
        END LOOP;
        DBMS_OUTPUT.PUT_LINE(line_text);
    END LOOP;
END;
/

-- 4. Built-in exceptions
DECLARE
    v_name  student.name%TYPE;
    v_num   NUMBER := 10;
    v_res   NUMBER;
BEGIN
```

```
    BEGIN                                       -- ZERO_DIVIDE
        v_res := v_num / 0;
    EXCEPTION
        WHEN ZERO_DIVIDE THEN
            DBMS_OUTPUT.PUT_LINE('ZERO_DIVIDE : cannot divide by zero');
    END;

    BEGIN                                       -- NO_DATA_FOUND
        SELECT name INTO v_name FROM student WHERE roll_no = 999;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('NO_DATA_FOUND : no student with roll 999');
    END;

    BEGIN                                       -- TOO_MANY_ROWS
        SELECT name INTO v_name FROM student WHERE dept_id = 10;
    EXCEPTION
        WHEN TOO_MANY_ROWS THEN
            DBMS_OUTPUT.PUT_LINE('TOO_MANY_ROWS : query returned many rows');
    END;
END;
/

-- 5. User-defined exception
DECLARE
    v_marks          NUMBER := 105;
    e_invalid_marks  EXCEPTION;
BEGIN
    IF v_marks < 0 OR v_marks > 100 THEN
        RAISE e_invalid_marks;
    END IF;
    DBMS_OUTPUT.PUT_LINE('Marks are valid');
EXCEPTION
    WHEN e_invalid_marks THEN
        DBMS_OUTPUT.PUT_LINE('User-defined exception : invalid marks '
                             || v_marks);
END;
/

-- 6. RAISE_APPLICATION_ERROR (error numbers -20000 to -20999)
DECLARE
    v_marks NUMBER := -5;
BEGIN
    IF v_marks < 0 OR v_marks > 100 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Marks must be between 0 and 100');
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error code    : ' || SQLCODE);
        DBMS_OUTPUT.PUT_LINE('Error message : ' || SQLERRM);
END;
/
```

Output

```
i = 1   running sum = 1
i = 2   running sum = 3
i = 3   running sum = 6
i = 4   running sum = 10
i = 5   running sum = 15

5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25

*
* *
* * *
* * * *

ZERO_DIVIDE : cannot divide by zero
NO_DATA_FOUND : no student with roll 999
TOO_MANY_ROWS : query returned many rows

User-defined exception : invalid marks 105

Error code    : -20001
Error message : ORA-20001: Marks must be between 0 and 100
```

Experiment 8

Aim: Programs using the creation of procedures, passing parameters IN and OUT of procedures.

Program

```
-- Experiment 8: Procedures with IN, OUT and IN OUT parameters
SET SERVEROUTPUT ON

-- 1. Procedure with IN and OUT parameters
CREATE OR REPLACE PROCEDURE get_student_details (
    p_roll   IN  student.roll_no%TYPE,
    p_name   OUT student.name%TYPE,
    p_marks  OUT student.marks%TYPE
) IS
BEGIN
    SELECT name, marks INTO p_name, p_marks
    FROM   student
    WHERE  roll_no = p_roll;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        p_name  := 'Not found';
        p_marks := NULL;
END get_student_details;
/

-- 2. Procedure that updates a table (IN parameters, one OUT parameter)
CREATE OR REPLACE PROCEDURE add_bonus (
    p_roll       IN  NUMBER,
    p_bonus      IN  NUMBER,
    p_new_marks  OUT NUMBER
) IS
BEGIN
    UPDATE student
    SET    marks = LEAST(marks + p_bonus, 100)
    WHERE  roll_no = p_roll
    RETURNING marks INTO p_new_marks;
END add_bonus;
/

-- 3. Procedure with an IN OUT parameter
CREATE OR REPLACE PROCEDURE square_it (p_num IN OUT NUMBER) IS
BEGIN
    p_num := p_num * p_num;
END square_it;
/

-- Calling the procedures from an anonymous block
DECLARE
    v_name   student.name%TYPE;
    v_marks  student.marks%TYPE;
    v_new    NUMBER;
    v_n      NUMBER := 7;
```

```
BEGIN
    get_student_details(101, v_name, v_marks);
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name || '   Marks: ' || v_marks);

    add_bonus(101, 5, v_new);
    DBMS_OUTPUT.PUT_LINE('Marks after bonus of 5: ' || v_new);
    ROLLBACK;                                -- keep the sample data unchanged

    square_it(v_n);
    DBMS_OUTPUT.PUT_LINE('Square of 7 (IN OUT): ' || v_n);

    get_student_details(999, v_name, v_marks);
    DBMS_OUTPUT.PUT_LINE('Roll 999 -> ' || v_name);
END;
/

-- Calling a procedure with EXECUTE (needs a bind variable for OUT)
VARIABLE b_name VARCHAR2(30)
VARIABLE b_marks NUMBER
EXECUTE get_student_details(108, :b_name, :b_marks);
PRINT b_name b_marks

-- Dropping a procedure
DROP PROCEDURE square_it;
```

Output

```
Name: Asha   Marks: 88
Marks after bonus of 5: 93
Square of 7 (IN OUT): 49
Roll 999 -> Not found

PL/SQL procedure successfully completed.

B_NAME
--------------------------------
Kumar

   B_MARKS
----------
        95

Procedure dropped.
```

Experiment 9

Aim: Programs using the creation of stored functions, invoking functions in SQL statements, and writing complex functions.

Program

```
-- Experiment 9: Stored functions, calling them in SQL, complex functions
SET SERVEROUTPUT ON

-- 1. Simple function : grade from marks
CREATE OR REPLACE FUNCTION get_grade (p_marks IN NUMBER)
RETURN VARCHAR2 IS
BEGIN
    RETURN CASE WHEN p_marks >= 90 THEN 'O'
                WHEN p_marks >= 80 THEN 'A'
                WHEN p_marks >= 70 THEN 'B'
                WHEN p_marks >= 60 THEN 'C'
                ELSE 'F' END;
END get_grade;
/

-- 2. Function that queries a table : average marks of a department
CREATE OR REPLACE FUNCTION dept_avg (p_dept IN NUMBER)
RETURN NUMBER IS
    v_avg NUMBER;
BEGIN
    SELECT AVG(marks) INTO v_avg FROM student WHERE dept_id = p_dept;
    RETURN ROUND(v_avg, 2);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
END dept_avg;
/

-- 3. Complex function : recursive factorial
CREATE OR REPLACE FUNCTION factorial (n IN NUMBER)
RETURN NUMBER IS
BEGIN
    IF n < 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Negative number not allowed');
    ELSIF n <= 1 THEN
        RETURN 1;
    ELSE
        RETURN n * factorial(n - 1);
    END IF;
END factorial;
/

-- 4. Complex function : prime number test using a loop
CREATE OR REPLACE FUNCTION is_prime (n IN NUMBER)
RETURN VARCHAR2 IS
    i NUMBER := 2;
```

```
BEGIN
    IF n < 2 THEN
        RETURN 'NO';
    END IF;
    WHILE i * i <= n LOOP
        IF MOD(n, i) = 0 THEN
            RETURN 'NO';
        END IF;
        i := i + 1;
    END LOOP;
    RETURN 'YES';
END is_prime;
/

-- Invoking the functions in SQL statements
SELECT name, marks, get_grade(marks) AS grade
FROM   student
ORDER BY roll_no;

SELECT dept_id, dept_avg(dept_id) AS average FROM department;

SELECT factorial(5) AS fact5,
       is_prime(17) AS prime_17,
       is_prime(15) AS prime_15
FROM   dual;

-- Invoking the functions from PL/SQL
BEGIN
    DBMS_OUTPUT.PUT_LINE('Factorial of 6 = ' || factorial(6));
    DBMS_OUTPUT.PUT_LINE('Grade for 73   = ' || get_grade(73));
END;
/

-- Functions can also be used in WHERE and ORDER BY clauses
SELECT name FROM student WHERE get_grade(marks) = 'O' ORDER BY name;

DROP FUNCTION is_prime;
```

Output

```
NAME            MARKS GRADE
---------- ---------- -----
Asha               88 A
Ravi               92 O
Meena              79 B
Kiran              65 C
Sita               90 O
Arjun              73 B
Divya              85 A
Kumar              95 O

   DEPT_ID    AVERAGE
---------- ----------
```

```
        10         84
        20       82.5
        30      83.33

     FACT5 PRIME_17 PRIME_15
---------- -------- --------
       120 YES      NO

Factorial of 6 = 720
Grade for 73   = B

NAME
----------
Kumar
Ravi
Sita

Function dropped.
```

Experiment 10

Aim: Programs using parameters in a CURSOR, FOR UPDATE cursor, WHERE CURRENT OF clause and cursor variables.

Program

```
-- Experiment 10: Cursor with parameters, FOR UPDATE cursor,
--                WHERE CURRENT OF, and cursor variables
SET SERVEROUTPUT ON

-- 1. Cursor with a parameter
DECLARE
    CURSOR c_dept (p_dept NUMBER) IS
        SELECT roll_no, name, marks
        FROM   student
        WHERE  dept_id = p_dept
        ORDER BY roll_no;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Department 10 ---');
    FOR r IN c_dept(10) LOOP
        DBMS_OUTPUT.PUT_LINE(r.roll_no || '  ' || r.name || '  ' || r.marks);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('--- Department 30 ---');
    FOR r IN c_dept(30) LOOP
        DBMS_OUTPUT.PUT_LINE(r.roll_no || '  ' || r.name || '  ' || r.marks);
    END LOOP;
END;
/

-- 2. FOR UPDATE cursor and WHERE CURRENT OF
--    (give 5 grace marks to students who scored below 70)
DECLARE
    CURSOR c_low IS
        SELECT roll_no, marks
        FROM   student
        WHERE  marks < 70
        FOR UPDATE OF marks;
    v_old NUMBER;
BEGIN
    FOR r IN c_low LOOP
        v_old := r.marks;
        UPDATE student
        SET    marks = marks + 5
        WHERE CURRENT OF c_low;
        DBMS_OUTPUT.PUT_LINE('Roll ' || r.roll_no || ': ' || v_old ||
                             ' -> ' || (v_old + 5));
    END LOOP;
    ROLLBACK;            -- keep the sample data unchanged
END;
/
```

```
-- 3. Cursor variable (REF CURSOR) opened for different queries
DECLARE
    TYPE t_cur IS REF CURSOR;
    cv       t_cur;
    v_name   student.name%TYPE;
    v_marks  student.marks%TYPE;
BEGIN
    OPEN cv FOR
        SELECT name, marks FROM student WHERE marks >= 90
        ORDER BY marks DESC;
    DBMS_OUTPUT.PUT_LINE('--- Toppers (marks >= 90) ---');
    LOOP
        FETCH cv INTO v_name, v_marks;
        EXIT WHEN cv%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_name || '  ' || v_marks);
    END LOOP;
    CLOSE cv;

    OPEN cv FOR
        SELECT name, marks FROM student WHERE dept_id = 20
        ORDER BY name;
    DBMS_OUTPUT.PUT_LINE('--- ECE students ---');
    LOOP
        FETCH cv INTO v_name, v_marks;
        EXIT WHEN cv%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_name || '  ' || v_marks);
    END LOOP;
    CLOSE cv;
END;
/
```

Output

```
--- Department 10 ---
101  Asha  88
103  Meena  79
107  Divya  85
--- Department 30 ---
104  Kiran  65
105  Sita  90
108  Kumar  95

Roll 104: 65 -> 70

--- Toppers (marks >= 90) ---
Kumar  95
Ravi  92
Sita  90
--- ECE students ---
Arjun  73
Ravi  92
```

Experiment 11

Aim: Programs using BEFORE and AFTER triggers, row and statement triggers, and INSTEAD OF triggers.

Program

```
-- Experiment 11: BEFORE / AFTER triggers, row and statement triggers,
--                INSTEAD OF triggers
SET SERVEROUTPUT ON

CREATE TABLE student_audit (
    audit_id   NUMBER PRIMARY KEY,
    roll_no    NUMBER,
    old_marks  NUMBER,
    new_marks  NUMBER,
    changed_on DATE
);
CREATE SEQUENCE audit_seq START WITH 1 INCREMENT BY 1;

-- 1. BEFORE INSERT statement-level trigger (fires once per statement)
CREATE OR REPLACE TRIGGER trg_before_insert_stmt
BEFORE INSERT ON student
BEGIN
    DBMS_OUTPUT.PUT_LINE('Statement trigger: BEFORE INSERT on STUDENT');
END;
/

-- 2. BEFORE INSERT/UPDATE row-level trigger : validation
CREATE OR REPLACE TRIGGER trg_before_row
BEFORE INSERT OR UPDATE ON student
FOR EACH ROW
BEGIN
    IF :NEW.marks < 0 OR :NEW.marks > 100 THEN
        RAISE_APPLICATION_ERROR(-20010, 'Marks must be between 0 and 100');
    END IF;
    :NEW.name := INITCAP(:NEW.name);
END;
/

-- 3. AFTER UPDATE row-level trigger : audit trail
CREATE OR REPLACE TRIGGER trg_after_update_row
AFTER UPDATE OF marks ON student
FOR EACH ROW
BEGIN
    INSERT INTO student_audit
    VALUES (audit_seq.NEXTVAL, :OLD.roll_no, :OLD.marks, :NEW.marks, SYSDATE);
    DBMS_OUTPUT.PUT_LINE('Row trigger: marks of ' || :OLD.roll_no ||
                         ' changed from ' || :OLD.marks ||
                         ' to ' || :NEW.marks);
END;
/
```

```
-- 4. AFTER DELETE statement-level trigger
CREATE OR REPLACE TRIGGER trg_after_delete_stmt
AFTER DELETE ON student
BEGIN
    DBMS_OUTPUT.PUT_LINE('Statement trigger: AFTER DELETE on STUDENT');
END;
/

-- 5. INSTEAD OF trigger on a join view
CREATE OR REPLACE VIEW student_dept_v AS
    SELECT s.roll_no, s.name, d.dept_name, s.marks
    FROM   student s JOIN department d ON s.dept_id = d.dept_id;

CREATE OR REPLACE TRIGGER trg_instead_insert
INSTEAD OF INSERT ON student_dept_v
FOR EACH ROW
DECLARE
    v_dept NUMBER;
BEGIN
    SELECT dept_id INTO v_dept FROM department
    WHERE  dept_name = :NEW.dept_name;
    INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (:NEW.roll_no, :NEW.name, v_dept, :NEW.marks);
END;
/

-- ---------------- Testing the triggers ----------------
INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (112, 'pooja', 10, 77);

UPDATE student SET marks = 80 WHERE roll_no = 112;

SELECT roll_no, name, marks FROM student WHERE roll_no = 112;
SELECT roll_no, old_marks, new_marks FROM student_audit;

INSERT INTO student (roll_no, name, dept_id, marks)
    VALUES (113, 'Rahul', 20, 150);

INSERT INTO student_dept_v VALUES (114, 'Sneha', 'ECE', 68);
SELECT * FROM student_dept_v WHERE roll_no = 114;

DELETE FROM student WHERE roll_no IN (112, 114);

-- Clean-up
COMMIT;
DROP TRIGGER trg_before_insert_stmt;
DROP TRIGGER trg_before_row;
DROP TRIGGER trg_after_update_row;
DROP TRIGGER trg_after_delete_stmt;
DROP TRIGGER trg_instead_insert;
```

Output

```
Statement trigger: BEFORE INSERT on STUDENT
1 row created.

Row trigger: marks of 112 changed from 77 to 80
1 row updated.

   ROLL_NO NAME              MARKS
---------- ---------- ----------
       112 Pooja              80       (name converted by INITCAP)

   ROLL_NO  OLD_MARKS  NEW_MARKS
---------- ---------- ----------
       112         77         80

Statement trigger: BEFORE INSERT on STUDENT
ORA-20010: Marks must be between 0 and 100

Statement trigger: BEFORE INSERT on STUDENT
1 row created.

   ROLL_NO NAME       DEPT_NAME       MARKS
---------- ---------- ---------- ----------
       114 Sneha      ECE                68

Statement trigger: AFTER DELETE on STUDENT
2 rows deleted.
```

Experiment 12

Aim: Create a table and perform the search operation on the table using indexing and non-indexing techniques.

Program

```
-- Experiment 12: Searching a table with and without an index
SET TIMING ON
SET LINESIZE 150

-- 1. Create a large table (200000 rows) using a row generator
CREATE TABLE emp_big AS
SELECT LEVEL                        AS emp_id,
       'Employee_' || LEVEL         AS emp_name,
       MOD(LEVEL, 50) + 1           AS dept_id,
       10000 + MOD(LEVEL * 37, 50000) AS salary
FROM   dual
CONNECT BY LEVEL <= 200000;

SELECT COUNT(*) FROM emp_big;

-- 2. Search WITHOUT an index : the optimizer must scan the whole table
EXPLAIN PLAN FOR
SELECT * FROM emp_big WHERE emp_name = 'Employee_150000';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

SELECT * FROM emp_big WHERE emp_name = 'Employee_150000';

-- 3. Create an index on the search column
CREATE INDEX idx_emp_name ON emp_big (emp_name);
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'EMP_BIG');

-- 4. Search WITH the index : the optimizer uses an index lookup
EXPLAIN PLAN FOR
SELECT * FROM emp_big WHERE emp_name = 'Employee_150000';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

SELECT * FROM emp_big WHERE emp_name = 'Employee_150000';

-- 5. Other kinds of indexes
CREATE UNIQUE INDEX idx_emp_id ON emp_big (emp_id);       -- unique
CREATE INDEX idx_dept_sal ON emp_big (dept_id, salary);   -- composite
CREATE INDEX idx_upper_name ON emp_big (UPPER(emp_name)); -- function

-- 6. List the indexes of the table
SELECT index_name, uniqueness FROM user_indexes
WHERE  table_name = 'EMP_BIG';

-- 7. Drop the indexes and the table
DROP INDEX idx_emp_name;
```

```
DROP INDEX idx_emp_id;
DROP INDEX idx_dept_sal;
DROP INDEX idx_upper_name;
DROP TABLE emp_big;
```

Output

```
Without the index - the plan contains:
| Id | Operation         | Name    |
|  0 | SELECT STATEMENT  |         |
|* 1 |  TABLE ACCESS FULL| EMP_BIG |     (all 200000 rows are scanned)

With the index - the plan contains:
| Id | Operation                           | Name         |
|  0 | SELECT STATEMENT                    |              |
|  1 |  TABLE ACCESS BY INDEX ROWID BATCHED| EMP_BIG      |
|* 2 |   INDEX RANGE SCAN                  | IDX_EMP_NAME |

Both queries return the same row:
    EMP_ID EMP_NAME           DEPT_ID     SALARY
---------- --------------- ---------- ----------
    150000 Employee_150000          1      10000

The indexed search reads only a few blocks, so its elapsed time (shown
by SET TIMING ON) is much smaller than the full table scan.

INDEX_NAME       UNIQUENESS
---------------- ----------
IDX_EMP_NAME     NONUNIQUE
IDX_EMP_ID       UNIQUE
IDX_DEPT_SAL     NONUNIQUE
IDX_UPPER_NAME   NONUNIQUE
```

Experiment 13

Aim: Write a Java program that connects to a database using JDBC.

Program (Java)

```
// Experiment 13: Java program that connects to a database using JDBC
// Compile : javac JdbcConnect.java
// Run     : java -cp .;ojdbc8.jar JdbcConnect   (Linux: use : not ;)
import java.sql.Connection;
import java.sql.DatabaseMetaData;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class JdbcConnect {
    public static void main(String[] args) {
        // Oracle connection details (change them for your system)
        String url      = "jdbc:oracle:thin:@localhost:1521:xe";
        String user     = "system";
        String password = "oracle";
        // For MySQL use: jdbc:mysql://localhost:3306/college

        try {
            // Step 1: load the JDBC driver
            Class.forName("oracle.jdbc.driver.OracleDriver");

            // Step 2: establish the connection
            try (Connection con =
                     DriverManager.getConnection(url, user, password)) {
                System.out.println("Connection established successfully.");

                DatabaseMetaData md = con.getMetaData();
                System.out.println("Database : "
                                   + md.getDatabaseProductName());
                System.out.println("Driver   : " + md.getDriverName());

                // Step 3: create a statement and run a query
                try (Statement st = con.createStatement();
                     ResultSet rs = st.executeQuery(
                         "SELECT roll_no, name, marks FROM student " +
                         "ORDER BY roll_no")) {

                    System.out.println("\nROLL  NAME     MARKS");
                    // Step 4: process the result set
                    while (rs.next()) {
                        System.out.printf("%-5d %-8s %d%n",
                            rs.getInt("roll_no"),
                            rs.getString("name"),
                            rs.getInt("marks"));
                    }
                }
```

```
            }   // Step 5: the connection is closed automatically
            System.out.println("\nConnection closed.");
        } catch (ClassNotFoundException e) {
            System.out.println("JDBC driver not found: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}
```

Output

```
Connection established successfully.
Database : Oracle
Driver   : Oracle JDBC driver

ROLL  NAME     MARKS
101   Asha     88
102   Ravi     92
103   Meena    79
104   Kiran    65
105   Sita     90
106   Arjun    73
107   Divya    85
108   Kumar    95

Connection closed.
```

Experiment 14

Aim: Write a Java program to connect to a database using JDBC and insert values into it.

Program (Java)

```
// Experiment 14: Java program to connect to a database using JDBC and
//                insert values into it
// Run : java -cp .;ojdbc8.jar JdbcInsert
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;

public class JdbcInsert {
    public static void main(String[] args) {
        String url      = "jdbc:oracle:thin:@localhost:1521:xe";
        String user     = "system";
        String password = "oracle";

        String sql = "INSERT INTO student (roll_no, name, dept_id, marks) "
                   + "VALUES (?, ?, ?, ?)";

        try (Scanner sc = new Scanner(System.in)) {
            Class.forName("oracle.jdbc.driver.OracleDriver");

            // read the values to be inserted
            System.out.print("Enter roll number : ");
            int roll = sc.nextInt();
            System.out.print("Enter name        : ");
            String name = sc.next();
            System.out.print("Enter dept id     : ");
            int dept = sc.nextInt();
            System.out.print("Enter marks       : ");
            int marks = sc.nextInt();

            try (Connection con =
                     DriverManager.getConnection(url, user, password);
                 PreparedStatement ps = con.prepareStatement(sql)) {

                con.setAutoCommit(false);        // control the transaction
                ps.setInt(1, roll);              // bind the parameters
                ps.setString(2, name);
                ps.setInt(3, dept);
                ps.setInt(4, marks);

                int rows = ps.executeUpdate();   // run the INSERT
                con.commit();                    // save the change
                System.out.println(rows + " row inserted successfully.");
            } catch (SQLException e) {
                System.out.println("Insert failed: " + e.getMessage());
            }
```

```
        } catch (ClassNotFoundException e) {
            System.out.println("JDBC driver not found: " + e.getMessage());
        }
    }
}
```

Output

```
Enter roll number : 120
Enter name        : Pavan
Enter dept id     : 10
Enter marks       : 82
1 row inserted successfully.

(Running it again with the same roll number gives:
 Insert failed: ORA-00001: unique constraint (SYSTEM.SYS_C0012345) violated)

Check in SQL*Plus:
SQL> SELECT * FROM student WHERE roll_no = 120;
   ROLL_NO NAME       DEPT_ID      MARKS
---------- ---------- ------- ----------
       120 Pavan           10         82
```

Experiment 15

Aim: Write a Java program to connect to a database using JDBC and delete values from it.

Program (Java)

```
// Experiment 15: Java program to connect to a database using JDBC and
//                delete values from it
// Run : java -cp .;ojdbc8.jar JdbcDelete
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;

public class JdbcDelete {
    public static void main(String[] args) {
        String url      = "jdbc:oracle:thin:@localhost:1521:xe";
        String user     = "system";
        String password = "oracle";

        String sql = "DELETE FROM student WHERE roll_no = ?";

        try (Scanner sc = new Scanner(System.in)) {
            Class.forName("oracle.jdbc.driver.OracleDriver");

            System.out.print("Enter roll number to delete : ");
            int roll = sc.nextInt();

            try (Connection con =
                     DriverManager.getConnection(url, user, password);
                 PreparedStatement ps = con.prepareStatement(sql)) {

                con.setAutoCommit(false);
                ps.setInt(1, roll);
                int rows = ps.executeUpdate();   // run the DELETE

                if (rows > 0) {
                    con.commit();
                    System.out.println(rows + " row(s) deleted.");
                } else {
                    con.rollback();
                    System.out.println("No student found with roll number "
                                       + roll);
                }
            } catch (SQLException e) {
                System.out.println("Delete failed: " + e.getMessage());
            }
        } catch (ClassNotFoundException e) {
            System.out.println("JDBC driver not found: " + e.getMessage());
        }
    }
}
```

Output

```
Enter roll number to delete : 120
1 row(s) deleted.

(If the roll number does not exist:
 Enter roll number to delete : 999
 No student found with roll number 999)
```